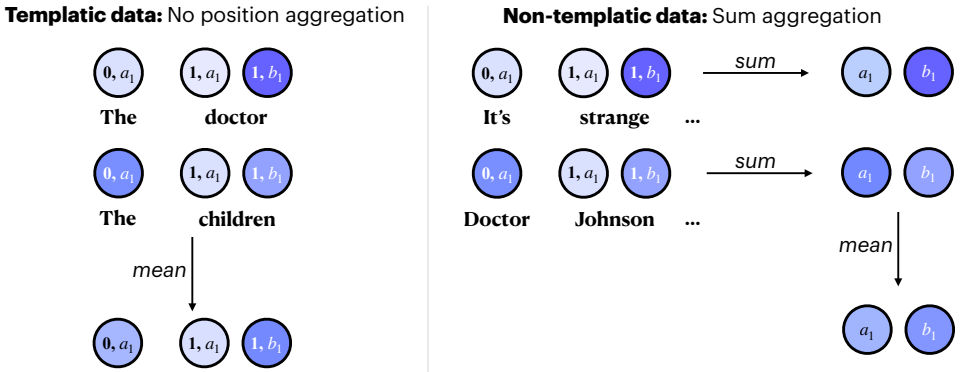

Figure 6: Aggregation of node/edge effects across examples (and sometimes, across token positions). Each feature is labeled as "token position, feature index." If we have templatic data, we preserve token position information, and treat the same features in different token positions as different features. If we have more general non-templatic data, we first sum across positions, and then take the example-wise mean of the position-aggregated effects.

Intuitively, this captures the indirect effect of $\mathbf{u}$ on m via the *direct* effect on $\mathbf{d}$, but excluding effects on $\mathbf{d}$ mediated by some other intermediate node $\mathbf{m}$.

As with nodes, we employ a linear approximation:

$$\hat{\text{IE}}(m; e; x_{\text{clean}}, x_{\text{patch}}) = \nabla_{\mathbf{d}} m\big|_{\mathbf{d}_{\text{clean}}} \nabla_{\mathbf{u},\text{stop}(\mathcal{M})} \mathbf{d}\big|_{\mathbf{u}_{\text{clean}}} (\mathbf{u}_{\text{patch}} - \mathbf{u}_{\text{clean}}) \tag{5}$$

where $\nabla_{\mathbf{u},\text{stop}(\mathcal{M})}\mathbf{d}$ denotes the gradient of $\mathbf{d}$ with respect to $\mathbf{u}$ when treating all $\mathbf{m} \in \mathcal{M}$ as constant. In practice, this is computed by applying stop-gradients to all intermediate nodes $\mathbf{m}$ during pytorch's backwards pass.

If $\mathbf{d}$ is an SAE error, then the naive approach to computing is expression involves performing d_{model} backwards passes; fortunately we can still compute the product in a single backwards pass as explained in §A.3.

A.2 Aggregating across token positions and examples

Figure 6 summarizes how we aggregate effects across examples (and optionally across token positions). For templatic data where tokens in matching positions play consistent roles (see §3.2, 3.3), we take the mean effect of nodes/edges across examples. In this case, we treat the same feature (or neuron) in different token positions as different nodes altogether in the circuit, each with their own separate effects on target metric m.

For non-templatic data (§4, 5), we first sum the effects of corresponding nodes/edges across token positions before taking the example-wise mean. This means that each feature appears in the circuit once, representing its effects at all token positions in an input.

A.3 Practical considerations

Here we review a number of tricks that we use to compute the quantities defined above efficiently. The backbone of our approach is to, given an activation $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ of some submodule for which we have an SAE, use the SAE to compute the quantities $f_i(\mathbf{x})$ and $\boldsymbol{\epsilon}(\mathbf{x})$ in (1), and then intervene in our model's forward pass to set

$$\mathbf{x} \leftarrow \sum_i f_i(\mathbf{x})\mathbf{v}_i + \mathbf{b} + \boldsymbol{\epsilon}(\mathbf{x}). \tag{6}$$

Even though $\mathbf{x}$ was already numerically equal to the right-hand side of (6), after the intervention the computation graph will incorporate the variables $f_i(\mathbf{x})$ and $\boldsymbol{\epsilon}(\mathbf{x})$. Thus, when we use Pytorch's autograd algorithm to peform backpropogation of downstream quantities, we will automatically compute gradients for these variables.

An alternative approach for computing gradients (which we do not use) is to simply run the model without interventions, use backpropogation to compute all gradients $\nabla_{\mathbf{x}} m$, and use the formulas

$$\nabla_{f_i} m = \nabla_{\mathbf{x}} m \cdot \mathbf{v}_i, \qquad \nabla_{\boldsymbol{\epsilon}} m = \nabla_{\mathbf{x}} m$$

which follow from the chain rule when m is any function of $\mathbf{x}$.

Stop gradients on SAE errors to compute SAE feature gradients. The natural way to compute the SAE error $\boldsymbol{\epsilon}(\mathbf{x})$ is by first using the SAE to compute $\hat{\mathbf{x}}$ and then setting $\boldsymbol{\epsilon}(\mathbf{x}) = \mathbf{x} - \hat{\mathbf{x}}$. However, if we take this approach, then after applying the intervention (6) we would have

$$\nabla_{f_i} m = \nabla_{vx^d} m \nabla_{f_i} \mathbf{x}^d = \nabla_{\mathbf{x}^d} m \nabla_{f_i} \left(\hat{\mathbf{x}} + \mathbf{x}^u - \hat{\mathbf{x}}\right) = 0$$

where $\mathbf{x}^d$ is the copy of $\mathbf{x}$ downstream of f_i in the computation graph, and $\mathbf{x}^u$ is the copy upstream of f_i. To fix this, we apply a stop gradient to $\boldsymbol{\epsilon}(\mathbf{x})$ so that $\mathbf{x}^d = \hat{\mathbf{x}} + \text{stopgrad}(\mathbf{x}^u - \hat{\mathbf{x}})$.

Pass-through gradients. Although the stop gradient from above solves the problem of vanishing gradients for the f_i, it interferes with the backpropogation of gradients to further upstream nodes. In order to restore exact gradient computation, we implement a pass-through gradient on the computation of our dictionary. That is, in the notation above, we intervene in the *backwards* pass of our model to set

$$\nabla_{\mathbf{x}^u} m \leftarrow \nabla_{\mathbf{x}^d} m.$$

Jacobian-vector products. Done naively, computing the quantity in (5) when $\mathbf{d}$ is an SAE errors would take $O(d_{\text{model}})$ backwards passes. Fortunately, one can use the following trick: when A is a constant $1 \times n$ matrix, $\mathbf{x} \in \mathbb{R}^m$, and $\mathbf{y} = \mathbf{y}(\mathbf{x}) \in \mathbb{R}^n$ is a function of $\mathbf{x}$, we have

$$A \nabla_{\mathbf{x}} \mathbf{y} = \nabla_{\mathbf{x}} (A\mathbf{y})$$

where the right-hand side is a $1 \times m$ Jacobian which can be computed with a single backward pass. Thus we can compute (5) with only two backwards passes by first computing $\nabla_{\mathbf{d}} m|_{\mathbf{d}_{\text{clean}}}$ and then computing $\nabla_{\mathbf{u}} \left(\nabla_{\mathbf{d}} m|_{\mathbf{d}_{\text{clean}}}\right)$ with another backwards pass, where the second $\nabla_{\mathbf{d}} m|_{\mathbf{d}_{\text{clean}}}$ is treated as a constant (e.g., by detaching it in Pytorch).

B Details on Sparse Autoencoders

B.1 Pythia-70M Sparse Autoencoders

B.1.1 Architecture

Following Bricken et al. (2023), our SAEs for Pythia-70M are one-layer MLPs with a tied pre-encoder bias. In more detail, our SAEs have parameters

$$W_E \in \mathbb{R}^{d_{\text{SAE}} \times d_{\text{model}}}, W_D \in \mathbb{R}^{d_{\text{model}} \times d_{\text{SAE}}}, \qquad b_E \in \mathbb{R}^{d_{\text{SAE}}}, b_D \in \mathbb{R}^{d_{\text{model}}}$$

where the columns of W_D are constrained to be unit vectors. Given an input activation $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$, we compute the sparse features activations via

$$\mathbf{f} = [f_1(\mathbf{x}) \quad \ldots \quad f_{d_{\text{SAE}}}(\mathbf{x})] = \text{ReLU}(W_E(\mathbf{x} - \mathbf{b}_D) + \mathbf{b}_E)$$

with the ReLU nonlinearity applied coordinatewise and reconstructions via

$$\hat{\mathbf{x}} = W_D \mathbf{f} + \mathbf{b}_D.$$

The feature vectors $\mathbf{v}_i \in \mathbb{R}^{d_{\text{model}}}$ are the columns of W_D.

B.1.2 Training

Fix a specific choice of activation in Pythia-70M, e.g. MLP output, attention output, or residual stream in a particular layer. Following Cunningham et al. (2024); Bricken et al. (2023) we train an SAE for this activation by sampling random text from The Pile (Gao et al., 2020) (specifically the

first 128 tokens of random documents), extracting the values $\mathbf{x}$ for this activation over every token, and then training our SAE to minimize a loss function

$$\mathcal{L} = \mathcal{L}_{\text{reconstruction}} + \lambda \mathcal{L}_{\text{sparsity}} = \|\hat{\mathbf{x}} - \mathbf{x}\|_2 + \lambda \|\mathbf{f}\|_1$$

consisting of a L2 reconstruction loss and a L1 regularization term to promote sparsity. This loss is optimized using a variant of Adam (Kingma & Ba, 2014) adapted to ensure that the columns of W_D are unit vectors (see Bricken et al. (2023) or our code for details). We use $\lambda = 0.1$ and a learning rate of 10^{-4}.

Following Nanda (2023), we cache activations from 10000 contexts in a buffer and randomly sample batches of size 2^{14} for training our SAE. When the buffer is half-depleted, we replenish it with fresh tokens from The Pile. We train for 120000 steps, resulting in a total of about 2 billion training tokens.

A major obstacle in training SAEs is *dead features*, that is, neurons in the middle layer of the SAE which never or rarely activate. We mitigate this by, every 25000 training steps, reinitializing features which have not activated in the previous 12500 steps using the same reinitialization procedure described in Bricken et al. (2023).

Finally, we use a linear learning rate warmup of 1000 steps at the start of training and after every time that neurons are resampled.

B.1.3 Evaluation

Here we report on various easy-to-quantify metrics of SAE quality. Note that these metrics leave out important qualitative properties of these SAEs, such as the interpretability of their features (App. F). Our metrics are:

- **Variance explained**, as measured by $1 - \frac{\text{Var}(\mathbf{x}-\hat{\mathbf{x}})}{\text{Var}(\mathbf{x})}$.
- Average **L1**, and **L0** norms of $\mathbf{f}$.
- **Percentage of features alive** as measured by features which activate at least once on a batch of 512 tokens.
- **Cross entropy** (CE) **difference** and **percentage of CE recovered**. The CE difference is the difference between the model's original CE loss and the model's CE loss when intervening to set $\mathbf{x}$ to the reconstruction $\hat{\mathbf{x}}$. We obtain percentage of CE recovered by dividing this difference by the difference between the original CE loss and the CE loss when zero-ablating $\mathbf{x}$. These CE losses are computed averaged over a batch of 128 contexts of length 128.

These metrics are shown in Tables 3–6. Note that we index residual stream activations to be the layer which *outputs* the activation (so the layer 0 residual stream is *not* the embeddings, and the layer 5 residual stream is the output of the final layer, immediately preceding the final decoder).

% Variance Explained	L1	L0	% Alive	CE Diff	% CE Recovered
96	1	3	36	0.17	98

Table 3: Embedding SAE evaluation.

B.2 Gemma-2-2B Sparse Autoencoders

For Gemma-2-2B, we use the Gemma Scope SAEs released by Lieberum et al. (2024), which are based on the Jump-ReLU architecture proposed by Rajamanoharan et al. (2024b). We use the SAEs of width 16384. There exist SAEs for the attention, MLP, and residual vectors for each of the 26 layers in the model. However, the attention and MLP SAEs are trained at different positions than in Pythia: attention SAEs are trained on the *input* to the out projection, and MLP SAEs are trained on the output of the LayerNorm following the MLP. The embedding SAEs are experimental and have a dictionary size of only 4000, so we do not use them in our experiments.